

codegraph_mcp_probe – guide

Revision: 1 **Last modified:** 2026-09-24T19:09:29Z

Source: `constitution/scripts/codegraph/codegraph_mcp_probe.py` (211 lines). Citations below are `codegraph_mcp_probe.py:<line>` .

Overview

A minimal, dependency-free MCP client. It starts ONE stdio MCP server command, speaks newline-delimited JSON-RPC 2.0 (protocol `2024-11-05`), performs `initialize` , `tools/list` and N `tools/call` requests, and prints a single JSON report (lines 5-8, 165-179, 204). It exists to prove with captured evidence, not by reading configuration, that a configured server answers a real query and how it holds the database: the access mode of each open `codegraph.db` descriptor is read from `/proc` (lines 9-13, 72-95).

The server is launched as `<--bin> serve --mcp [--extra-arg ...]` with the working directory set to `--project` (lines 18, 128-130).

Prerequisites

- `python3` , standard library only (line 35).
- Linux `/proc` for `db_fd_modes` , `rss_kb` and `threads` (lines 75, 101); where `/proc/<pid>/...` is unreadable those fields come back empty or `null` (lines 78-79, 107-108). UNCONFIRMED: behaviour on non-Linux hosts was not run.
- A server command that speaks MCP over stdio (default `codegraph` , line 58).
- `--project` must be a FIXTURE or a project the caller has verified is safe to open: the server opens that project's database (lines 19-21, 32-33). Never point it at a live index without that check.

Usage

```
codegraph_mcp_probe.py --project DIR [--bin CMD] [--env K=V]...  
  [--tool NAME] [--query TEXT] [--expect SUBSTR] [--calls N]  
  [--hold SECONDS] [--timeout SECONDS] [--stderr-file PATH]  
  [--list-only] [--extra-arg ARG]...
```

Flag	Default	Meaning	Source
<code>--project DIR</code>	required	Working directory and <code>rootUri</code> of the server	lines 57, 114, 130, 164
<code>--bin CMD</code>	<code>codegraph</code>	Server executable; run as <code>CMD serve --mcp</code>	lines 58, 128
<code>--env K=V</code>	none	Extra environment entry, repeatable; merged over the inherited environment, later entries win	lines 59, 116-121
<code>--tool NAME</code>	<code>code-graph_search</code>	Tool name for <code>tools/call</code>	line 60
<code>--query TEXT</code>	<code>compute_total</code>	Sent as <code>arguments: {"query": TEXT}</code>	lines 61, 179
<code>--expect SUBSTR</code>	the query text	Substring that must appear in every answer	lines 62, 176
<code>--calls N</code>	1	Number of <code>tools/call</code> requests	lines 63, 177
<code>--hold SECONDS</code>	0.0	Sleep this long after the probes, before the server is shut down	lines 64, 188-189
<code>--timeout SECONDS</code>	90.0	Wait limit for EACH response (initialize, tools/list, every call), not a total	lines 65, 141-160
<code>--stderr-file PATH</code>	temp file	Where the server's stderr goes	lines 66, 122-127
<code>--list-only</code>	off	Skip <code>tools/call</code> ; report <code>initialize</code> + <code>tools/list</code> only	lines 67, 175
<code>--extra-arg ARG</code>	none	Appended after <code>serve --mcp</code> , repeatable (not in the header usage line)	lines 68, 128

`-h / --help` is provided by `argparse ( add_help=True , line 56)`.

Inputs

Only the flags above; nothing is read from stdin. The environment is inherited by the server (line 116).

Outputs

ONE JSON line on stdout (line 204). Fields (lines 162-187):

Field	Meaning
<code>init_ok</code>	<code>true</code> when the <code>initialize</code> reply contained <code>result</code>
<code>server</code>	<code>result.serverInfo</code> of the <code>initialize</code> reply
<code>tools</code>	names from <code>tools/list</code>
<code>calls ,</code> <code>calls_ok</code>	requested vs. answered-as-expected count
<code>last</code>	first 300 characters of the last call's text (set only if a call ran)
<code>pid</code>	server PID
<code>db_fd_modes</code>	sorted list, one entry per open descriptor whose link target ends in <code>/codegraph.db : "r"</code> when <code>flags & 3 == 0</code> (O_RDONLY), else <code>"rw"</code> (lines 85-95)
<code>rss_kb ,</code> <code>threads</code>	<code>VmRSS</code> and <code>Threads</code> from <code>/proc/<pid>/status</code> (lines 98-109)
<code>stderr_path</code>	file holding the server's stderr

A call counts as OK when `--expect` (or the query) occurs in the joined `content[].text` AND `isError` is falsy (lines 182-185).

If the server cannot be spawned, the output is instead `{"init_ok": false, "error": "spawn failed: ...", "stderr_path": ...}` and the exit code is 1 (lines 132-134).

Exit codes

Code	Meaning	Source
0	Initialized AND every requested call was OK (<code>calls_ok == calls</code>); with <code>--list-only</code> : initialized	lines 205-207
1	Spawn failed, initialization failed, or any call failed the expectation / returned <code>isError</code> / timed out	lines 133-134, 205-207
2	Usage: <code>--project</code> not a directory, or <code>--env</code> value without <code>=</code> ; also argparse errors (missing <code>--project</code>)	lines 50-52, 114-119

Edge cases

- `--calls 0` without `--list-only` exits 0 once initialized, because `calls_ok == calls == 0` (line 207).
- If initialization fails, no calls are attempted (line 175) and `calls_ok` stays 0, so the exit is 1 unless `--calls 0` .
- Responses whose `id` does not match the awaited id (for example server notifications) are discarded, and undecodable lines are skipped (lines 148-153).
- EOF or timeout on the pipe yields `{"error": "eof"}` / `{"error": "timeout"}` for that request, which shows up as `init_ok: false` or a failed call (lines 157-160).
- `db_fd_modes` is sampled after the calls and before `--hold` , while the server is still alive (lines 186-189). An empty list means no descriptor on a `.../codegraph.db` path was found at that moment.
- `--stderr-file` is opened `wb` , so an existing file is truncated (line 124). Without it a `cg_mcp_probe_err.*` file is created under the temp directory and left in place (line 126).

Internal behaviour

1. Validate `--project` , build the environment, open the stderr sink (lines 113-127).
2. `Popen` the server with piped stdin/stdout, unbuffered (lines 128-131).
3. `initialize` (id 1, `capabilities.roots` , `clientInfo.name` = `codegraph_mcp_probe`), then the `notifications/initialized` notification, then `tools/list` (id 2) (lines 165-174).
4. Unless `--list-only` , send `tools/call` with ids `10 + i` (lines 175-185).
5. Sample descriptors and process stats, optionally hold (lines 186-189).
6. Shutdown in `finally` : close stdin, wait 10 s, `terminate` , wait 5 s, `kill` (lines 190-203), then print the report.

Related

Guide / file	Relation
<code>codegraph_mcp_preflight.sh</code>	Calls this probe (<code>code-graph_mcp_preflight.sh:37</code>)
<code>tests/test_mcp_readonly.sh</code> , <code>tests/test_codegraph_mcp.sh</code>	Use it as <code>PROBE</code> (<code>test_mcp_readonly.sh:46</code> , <code>test_code-graph_mcp.sh:27</code>)
owed_report	Reports whether this tool has tests and a guide
<code>runner_patches/mcpro1.py</code>	Read-only MCP runner patch named in the header cross-refs (line 37)

Header line 36 also cites `codegraph_mcp_runner.py` ; UNCONFIRMED: no such file exists in `scripts/codegraph` on the verification date.

Last verified

2026-09-24T19:09:29Z — ran from `scripts/codegraph` against a throwaway fixture (a stub MCP server script and an empty `codegraph.db` in a scratch directory; no real `codegraph` binary was started):

- `--calls 2 --extra-arg=--x` : JSON with `init_ok: true` , `calls_ok: 2` , `tools: ["codegraph_search"]` , `last: "found compute_total"` , `db_fd_modes: ["r"]` ; the stub's stderr file recorded `args=['serve', '--mcp', '--x']` ; exit 0.
- `--list-only` : exit 0. `--expect NOPE` : exit 1.
- `--bin <missing path>` : `{"init_ok": false, "error": "spawn failed: ..."} , exit 1.`
- Missing directory, `--env NOEQ` , and no `--project` : exit 2 each.
- `--calls 0` : exit 0.